

★CCAR-P 形式: 全 63 問・120 分・720 点 (72%) で合格。「複数選択 (select TWO/THREE)」と「シナリオ・マッチング」が本試験特有の形式なので、設問で「**複数選択**」を必ず先に読むこと。紛らわしい選択肢は「何もしない」「表明された制約と矛盾する」など**理由に依り**なので**読み込み**が効く。API の細部ではなく判断力を問う試験で、実務経験がそのまま通用する。時間は十分 (60 分で完遂した報告あり)。

D6 導入ライフサイクル 5 フェーズ ★ (最大弱点)

導入は 5 つのフェーズで進む。各フェーズで「何をするか」「いつ次へ進めるか」を理解する。

- ① **Discovery (発見)**: ステークホルダーとの「構造化された聞き取り (会話でなく聞き取り)」。相手のビジネス目標を聞き、その背後にある**隠れた制約**を設計可能な要件に翻訳する。相手が「シームレスに感じてほしい」と言えば、それは要件でなく「もっと質問が必要」という合図。レイテンシ上限・失敗時の挙動・統合の前提を追及質問で具体化する。出力は**翻訳テーブル** (発言・意味する制約・必要な決定・文書化された前提)。
- ② **Design (設計)**: 翻訳した要件をアーキテクチャ決定に変える。トレードオフを提示して、ステークホルダーが**決定できる**ようにする (自分で勝手に判決を下すのでない)。
- ③ **Handoff (引継ぎ)**: 決定を文書化する。読者は**3 種類** (後任エンジニア・コンプライアンス監査人・数ヶ月後に戻る自分)。1 人だけ向けなら不完全。棄却された代替案と「なぜその決定か」の根拠が、決定こと自体と同じくらい重要。
- ④ **Monitoring (監視)**: 本番の異常シグナルごとに「誰が、どう動くか」を決めた表 (ガバナンス表) を運用する。ダッシュボードで数値を見るだけでは「監視」と呼べず、**シグナル→担当者→行動**が繋がって初めて機能する。
- ⑤ **Iteration (反復)**: **カレンダー駆動**の定期レビュー。出力監査・データ居住性の再確認・ルール更新を、エラー有無に関わらず定期的に行う。

D6 重要な区別 (混同が誤答の原因)

- Monitoring ≠ Iteration**: Monitoring は本番シグナルのその都度のトリアージ (イベント駆動)。Iteration は定期カレンダーでルール自体を見直す (別物)。
- Monitoring ≠ Logging**: Monitoring は SLI 指標を計装し「事前にドリフトを捉える」。Logging は事後に記録から再構築する (ユーザー苦情の後)。offline eval やサンプルレビューは**ライブドリフトを捉えない**。
- 前提 ≠ 要件**: 要件はステークホルダー発言に遡れる。前提は誰も言及しなかったもの。**遡元不能な前提が最も危険** (誰も決定したことを覚えていない)。
- トレードオフ 3 要素**: 得るもの／犠牲／**逆転コスト** (後で決定を覆す cost)。逆転コストが最もスキップされ、会議を変える要素。規制環境では 4 つ目に「コンプライアンス姿勢への影響」を加える。
- フィードバックループ**: ライフサイクル全体で前提を**再検証し続ける** (recurring)。一度きりのキックオフ承認でない。要件を一度集めて二度と確認しないとドリフト失敗になる。

D5 ガバナンス・安全性 3 層 ★ (混同注意)

安全性は単一の設定でなく「3 層のコントロール」。各層が異なる質問に答えるので、1 箇所のフィルタで他をカバーできない。

- 第 1 層: 予防ガードレール (発生前に block)**: 入力スクリーニング (モデル呼出前)・出力スクリーニング (ユーザ到達前)・**ツール呼出承認** (副作用 action 前) の 3 点。ツール承認は**ほぼ常に決定論的** (許可 list+ID+scope) で、監査可能・再生可能。出力スクリーニングは「テキストを判定する」もので action は止めないで、副作用は実行**前**に別承認が要る。
- fail-CLOSED が原則**: 自前のコントロールは障害時に「止める」。fail-open (通す) は「保護の錯覚」を与えつつ何も防がないので、コントロール無しより**悪い**。Anthropic 内蔵セーフティは別物で、構成不可・fail-open しない。
- 第 2 層: Human-in-the-loop (レビュー routing)**: 人に回す基準は「ボリューム」でなく**ステークス**。ステークスは**可逆性+誤決定 cost**。**信頼度**は 3 番目の変数: routing する**量**を変えるが、ステークス自体は変えない。**較正が必要** (自信を持って間違えうる)。低信頼度 × 不可逆/高 cost → 事前承認。高信頼度 × 可逆 × 低 cost → 通す。
- 全件 review は同意疲労**で無意味化 (読まずに click)。レビューアには**入力+出力+flag 理由**を提示。
- 第 3 層: 監視・監査 (決定 logging)**: 全自動決定を log し、事後説明・オンデマンド再構築を可能にする。**log ≠ routing** (log=事後説明、routing=誤決定の発効阻止)。

D5 責任スケールリング・公平性 ★

- 責任あるスケールリング=能力連動**: モデル能力が上がり・action 範囲が広がると、制御も強化し **HITL gate を再評価**する。1 年前の checklist は自動移行しない。safety は一度の sign-off で終わらない。
- バイアス ≠ 公平性**: バイアスは**測定された歪み (検出)**。公平性は**規範+選ばれた定義+執行 (解決)**。検出は作業の**開始で終了でない**。本番 drift が歪みを再導入するので、**サブグループ別の継続監視**が要る。集計精度は不均等な影響を隠す。
- error budget=SLO 由来**: 厳しい内部目標 (SLO) から算出される予算。**SLA (契約)/SLI (生指標) ではない**。契約違反前に反応余裕を与える。
- success criteria: 多次元・数値閾値・代表 set・部分点採点**。単一の全体精度% や都合の良い sample は弱 pattern。

D1 パターン選択 ★ (過剰設計を避ける)

Claude の関与の「形」は 4 pattern。予測可能性とモデル自律性の 2 軸で選ぶ。

- Augmented LLM (強化された単一呼出)**: 1 回の境界ある task。手順は 1 pass で制御フローは分岐しない。最速・最安。単発生成・tool 無し/軽微な時に使う。
- Workflow (ワークフロー)**: 手順が**既知・毎回同一**。code で各 step を配線し、制御フローは code 内 (予測可能・log・test 容易)。step を事前に決定でき、誤謬 cost が現実的な時に使う。
- Autonomous agent (自律 agent)**: 実行時にモデルが次手順を**動的決定**する。制御フローはモデル内。手順を事前列挙**できない**時に限る。本番では工具・予算・権限・停止基準で縛る (これらは必須)。
- Multi-agent (マルチ agent)**: orchestrator が作業を**分解・委任・合成**し、自らは実行しない。各 sub-agent は独立 context で一部を担う。作業が本当に 1 context を超える時のみ。

D1 過剰設計禁止・workflow 細形 ★

- 過剰設計禁止**: 手順が既知なら **workflow** (自律 agent でない)。multi-agent の複雑さは「**真に異なる専門家**」か「**並列独立 sub-task**」の時のみ正当化。**最も単純な pattern**を選び、測定が不足を示した時のみ昇格する。
- workflow 4 細形: chaining** (線形順次・各段階の出力を次が消費) / **routing** (分類器が入力種別で下流 path を切替) / **parallelization** (独立 sub-task を同時実行→集約/投票) / **evaluator-optimizer** (生成→評価→改訂 loop、基準充足か再試行上限迄)。
- 5 要素 framework** (順に見て、最初に pattern を除外した要素が決定的): 予測可能性／誤謬 cost / 観測性 / latency 予算 / 量での cost。
- router 過負荷に注意**: 工具選択精度が選択肢増で低下する時は、1 agent に詰め込むのではなく**領域特化 agent**に再編する。

D1 分解・委譲の原則

- 全 job は **3 所有者**: Claude (言語理解・草案) / 既存 system (決定論的 rule・真実の source) / 人 (判断・例外・承認)。
- Claude への過剰割当が最高 cost の初期 miss**。決定論的 check を model に渡すと、大部分は正しいが稀に間違え、監査まで不可視。
- 委譲の判定: **可逆性・stake・説明責任**の 3 変数。

D3 統合・エントリーポイント ★(最大配点 19%)

- 統合層は Claude を企業 system に埋め込む部分。誤ると本番で発覚する。
- ・ **コンプライアンス制約が先**: コンプライアンスが、cost や技術の好みの前に entry point を排除する(他決定の前提)。統合の 5 layer 順: compliance → identity/SSO → authz/policy → data/PII → observability/audit。
  - ・ **RAG は静的・緩変知識向き**: manual・規制文など。前処理で chunking+indexing し、query 時に関連 chunk を取得する。変更が少ない知識に適する。
  - ・ **RAG は live state に不適**: 在庫・注文 status・価格など刻々変わる値では、取得した snapshot が陳腐化する(10:00 の正解が 10:05 には誤り)。live state は **tool 呼出で真実の source を直接 lookup** する(知識取得と live state を分離するのが定石)。
  - ・ **統合 interface の選択**: **Direct API**(完全制御・retry/stream/error 全自前で実装重) / **SDK**(便利層・orchestration は自前) / **Agent SDK**(managed loop・多 turn 行動向き・単発には不適) / **MCP**(統合層を orchestration から分離・1 server/1 owner/1 version) / **Claude Code**(開発者向け・製品 backend には不適)。
  - ・ **統合選択の判別**: MCP=多 consumer/発見可能/再利用。Direct API=決定論的・密 scope 呼(発見層不要)。Agent-to-agent=組織 trust 境界越え(各 agent が自 system を媒介)。
  - ・ **identity=server 側注入**: user が主張不可(偽造防止)。server が role や承認済 doc set を system prompt に注入し、既存 authz model が Claude 層も **支配**(bypass 禁止)。
  - ・ **最小権限 tool**: 不要 tool は削除。orchestrator-worker は sub-agent 毎に scope。取得 doc/tool 出力は user 入力とは別の **第 2 の injection 経路**なので、user 入力と同様に screen する。

D3 retrieval・chunking の要点

- ・ **retrieval vs data shape: exact ID** → lexical 勝ち(embedding は滲む)・**自然言語**→ semantic 勝ち。**hybrid** が query 形状に戦略を照合。
- ・ **chunking**: 固定 size は定義/相互参照を切断する。**文書構造に整合**させた chunk + linkage metadata。
- ・ **stale embedding 防止**: 検証済 re-index pipeline + version-aware retrieval(古 chunk が滞留しても serving を阻止)。

規制・コンプライアンス用語 ★(用語未習＝弱点)

- 規制は「結果」を述べるだけで技術は自分で決める。各義務を **control+owner+証拠**に落とす。
- ・ **HIPAA**: 米国の健康データ (PHI)。署名済 **BAA(Business Associate Agreement)** が必須。BAA は「各当事者の安全策・責任」を定める契約で、これが無ければ**技術的にどう作っても非準拠**。BAA は製品単位でなく**構成単位**で結ぶ。
  - ・ **GDPR**: EU 個人データ。**DPO(Data Protection Officer)**(データ保護責任者)。DPA=データ処理契約。越境転送は有効な転送機構があれば合法 (EU pin でなくてもよい)。
  - ・ **FedRAMP**: 米国政府 **workload**。必要な impact level が必要。直接 API Enterprise は**非承認**(Claude for Government / Bedrock GovCloud / Vertex Assured 等、文書化済み承認 route のみ)。
  - ・ **データ居住性 (data residency)**: 実行 region を**固定**。処理・保存だけでなく、**logs/caches/監視/保持 path も境界内**に留める。ログが別 region に書かれて居住性違反になるのが監査落ちの定番。
  - ・ **各義務=control+owner+証拠 artifact**(3 点セット)。証拠が無ければ監査で不可視(コントロールが動いたという表明だけは監査人が受け入れない)。
  - ・ **「準拠 route 選択 ≠ コンブラ証明」**: route 選択は前提条件であって証拠でない。設定が drift すると設計時は正しかった居住性が本番で静かに偽りになる。

D4 評価・失敗判別 ★(model/prompt/hallucination の判別)

- 品質低下の「原因」を正しく分類しないと、誤った修正が響く。
- ・ 指標: accuracy / **hallucination rate**(unsupported 捏造の割合) / latency **p95**(中央値でなく末尾 tail) / cost-per-interaction。
  - ・ **model mismatch**: モデルが task に必要な**能力を持たない**(能力不足)。fix=**model 切替**。より強い model で同じ task が通れば能力起因と確定。
  - ・ **prompt failure**: モデルは**能力がある**が prompt が正しい挙動を引き出せない。fix=**prompt 修正**。**強 model でも同 prompt で失敗**すれば prompt 起因。
  - ・ **hallucination**: 流暢だが**事実 unsupported**・捏造。fix=grounding/RAG/citation。確信度は妥当性でない(同じ流暢さで間違える)。
  - ・ **eval=合格基準**: 変更が改善か回帰かを**本番前に判定**。**1 変数ずつ変更**(bundled は帰属破壊)。trace 分離で retrieval 層 vs model 層を診断。
  - ・ **eval dataset 設計**: **real query**(真の入力分布)+**構築 edge case**(破綻面)。規制案件は **golden set**(専門家 label・典型+高 risk)+**adversarial**(攻撃下で境界保持)。
  - ・ **LLM judge**: 主観 quality at scale は judge model が全出力に rubric 適用。**定期 human 校正**で judge を正直に保つ。
  - ・ **trace 診断**: code/model/prompt 不変で quality 低下なら原因は**流れる物**(入力分布)。**drift 分析**(入力変化か?)で retrieval 層 vs model 層を分離診断。
  - ・ offline 合格 ≠ 本番合格(分布が転移しない)。**canary**で blast radius を制限し、A/B で本番証拠を得る。

D2 context・cache

- ・ **prompt cache**: **固定 prefix**(system+tools+KB) を先・**動量**(per-user/time) を最後。動量を先頭に置くと全 prefix が一意になり **hit rate ゼロ**。
- ・ cache write=**割増**・cache read=**大幅割引**。**TTL 限定**(永続でない)。
- ・ **context window=入出力共有予算**(入+出>上限=拒否)。
- ・ **modular prompts/Skills**: 手順/template の **on-demand load**(全積みでない)。重要規則は**先頭 or 末尾**に置く(中間埋もれで喪失 = positional attention)。
- ・ context window は**データ統治境界でない**(access control は別層)。

判断トラップ ★(紛らわしい選択肢の型)

- ・ 干渉肢の 3 型: **「不要な overhead を加える」**(過剰設計) / **「何もしない・誤り容認」** / **「表明制約と矛盾」**(例: 高影響 action を ungated 残す)。
- ・ **trade-off は両辺を定量化**(教条でない)。SLA に対し余裕があれば採用+監視。例: 7pt 精度 vs +500ms が SLA 3s なら**余裕**→採用。
- ・ **「新 model ≠ 自 task で良い」** → 全 eval 群 +**canary**。
- ・ **「治療前に診断」**: 先に prompt を変えない(真因隠蔽+稼働挙動の不安定化 risk)。trace で先に原因を特定。
- ・ **指示=security 境界でない**→ system 層で access control(最小権限)。**capability ≠ injection 免疫**。
- ・ 証拠基盤の伝達: quality/rework/規制露出 vs 削減額を**量化→推奨→説明責任者が決定**(独断/隠蔽でない)。

MCP・orchestration 安全 ★

- ・ **障害非対称**: orchestrator 障害=**回復不能**(全 run 失敗・部分作業が取り残し)。sub-agent 障害=**回復可能**(retry/reroute/flag)。
- ・ 設計: sub-agent 作業は **idempotent+再試行可能・orchestrator 状態を保護・共有 trace ID**で end-to-end 再構築。
- ・ **context 分離**: 各 sub-agent は **clean context+tool access**を task に scope(信頼階層・最小権限)。
- ・ 合成時の **coverage check**: 返却結果数=配布 unit 数でなければ差を flag(不完全作業の上の自信ある合成が最危険)。